

Docker Swarm example1: Details (2025-05-15)

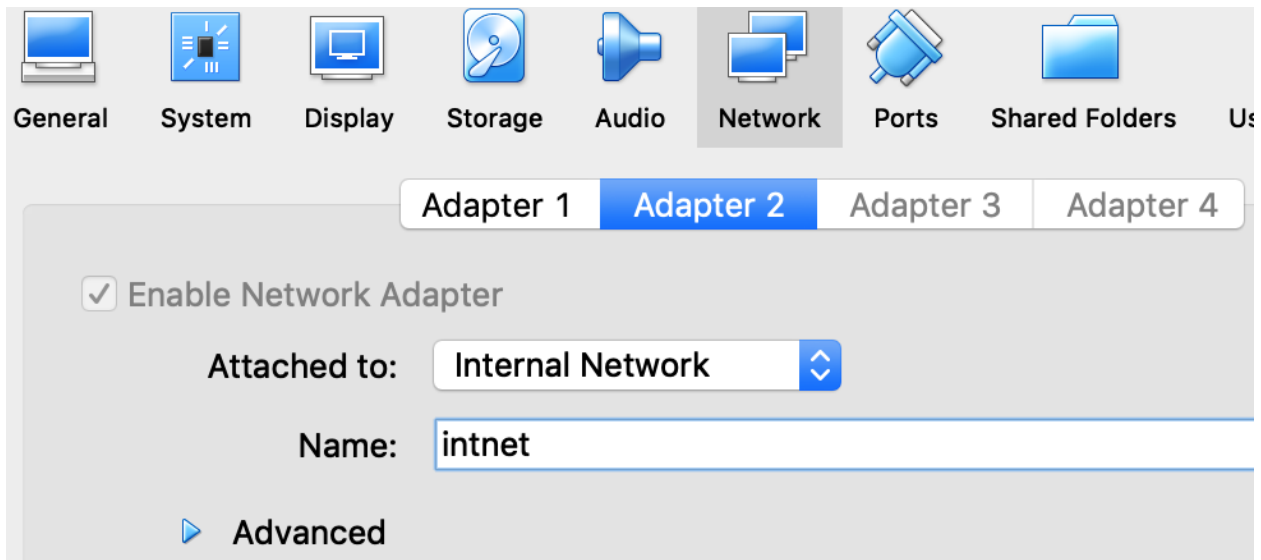
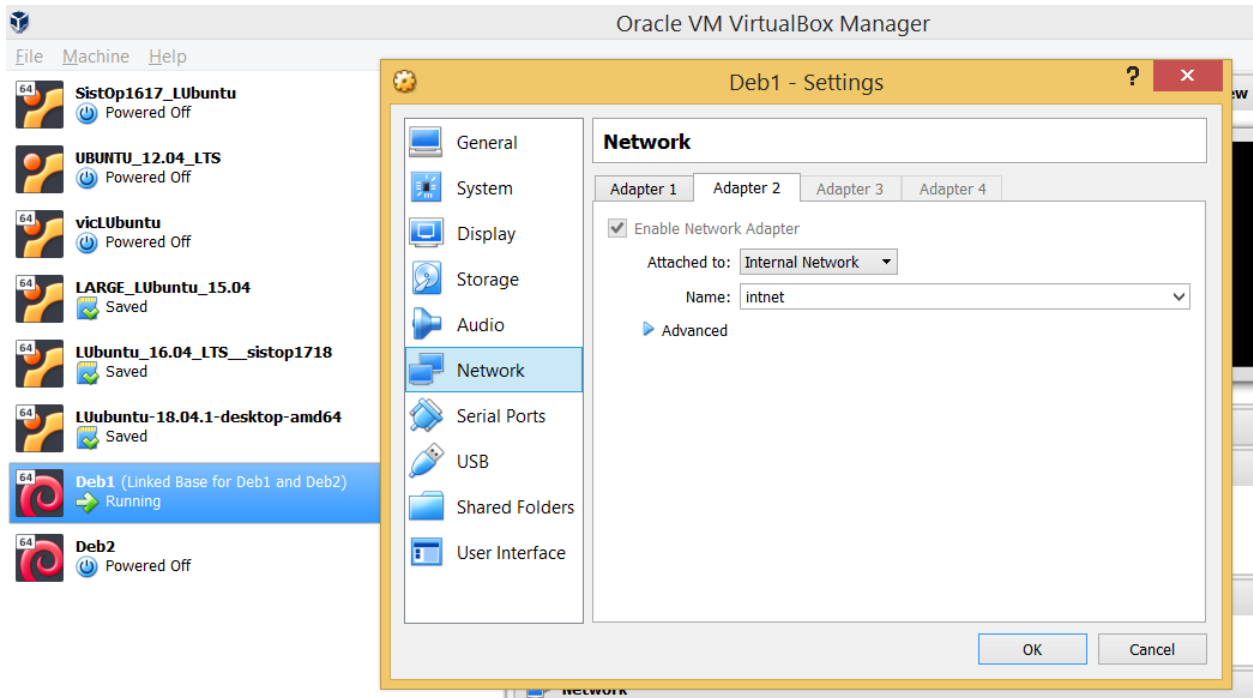
NB: Questo file è contenuto nella directory:
/cygdrive/c/DIDATTICA/SystemsIntegration_AmministrazioneDiSistemi/DISENSE/SwarmOnVirtualbox/
Questo esercizio funziona anche con le due VM create con Vagrant,
quelle che stanno nella directory
/cygdrive/c/DIDATTICA/SystemsIntegration_AmministrazioneDiSistemi/DISENSE/VM-swarm/

0. INSTALLING VMs

```
virtualbox in windows
deb1 in virtualbox
network adapter 1: NAT
network adapter 2:
    Attached to:    Internal Network
    Name:           intnet
```

I created two virtual machines running Debian in VirtualBox.

I created the first virtual machine, called Deb1.
In that virtual machine I created a first network card (Adapter 2) of type NAT. The hosted os will see that NIC as enps03.
In that virtual machine I added an additional network card (Adapter 2) of type "Internal Network" and name intnet, setup for Internal Networking. The hosted os will see that NIC as enps08



I install Debian 4.9.168-1 (2019-04-12) x86_64 GNU/Linux.
 The debian systems sees two network adapter:
 the first one (enp0s3) is the NAT interface.
 the second one (enp0s8) is the Internal interface.

I configured the Internal interface so that it works on the network 10.133.7.0/24. I assigned the static IP address 10.133.7.101 and netmask 255.255.255.0 to that enp0s8 interface.
 To do so I edited the file /etc/network/interfaces.d/setup that is included in the file /etc/network/interfaces adding:

```
auto enp0s8
iface enp0s8 inet static
address 10.133.7.101
netmask 255.255.255.0
# gateway 10.133.7.99
# dns-domain
# dns-nameservers 130.136.1.110 8.8.8.8
```

This causes the o.s. does not manage automatically that interface.

I configured debian for login automatic of the vic user, editing the
vi /etc/lightdm/lightdm.conf

I installed in debian some packages:

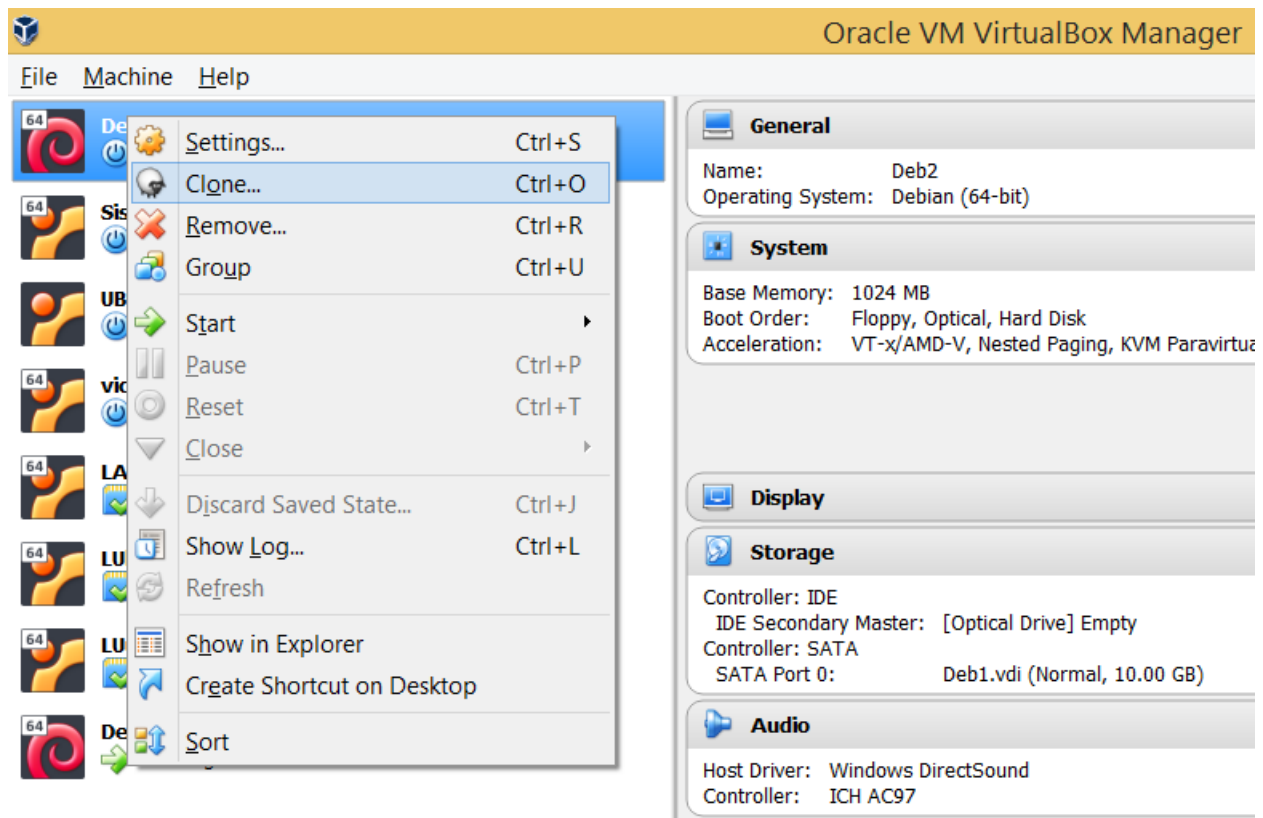
```
sudo apt install bridge-utils traceroute netcat net-utils jq
```

NB: jq is a lightweight and flexible command-line JSON processor

I installed in Debian the docker and docker-compose packages:

- ✓ sudo apt-get update
- ✓ sudo apt install apt-transport-https ca-certificates curl
software-properties-common
- ✓ curl -fsSL https://download.docker.com/linux/debian/gpg | sudo
apt-key add -
- ✓ sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/debian \$(lsb_release -cs)
stable"
- ✓ sudo apt-get update
- ✓ sudo apt-get install docker-ce docker-ce-cli containerd.io
- ✓ sudo apt-get install docker-compose
- ✓ sudo systemctl status docker

I cloned the first debian machine, selecting the Deb1 machine,
pressing the right mouse button, selecting clone.

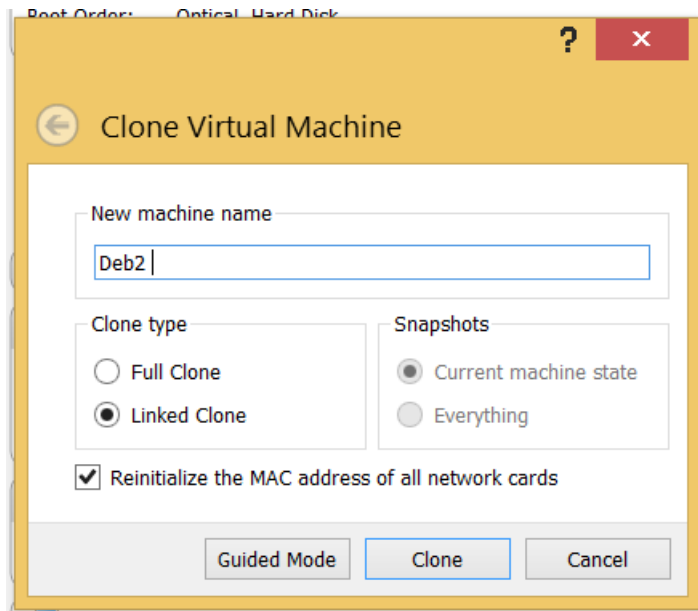


In the opened new window i selected "Expert mode", than I selected the following options: "Linked clone" and "Reinitalize the MAC address of all the network card". I assigned the new name: Deb2.

I started the new virtual machine Deb2 and I modified its network address of the enp0s8 interface.

To do so I edited the file /etc/network/interfaces.d/setup that is included in the file /etc/network/interfaces adding the following lines:

```
auto enp0s8
iface enp0s8 inet static
    address 10.133.7.102
    netmask 255.255.255.0
# gateway 10.133.7.99
# dns-domain
# dns-nameservers 130.136.1.110 8.8.8.8
```



Finally, I changed the hostname, from debian1 to debian2.

To do so, I run the following command:

```
sudo hostnamectl set-hostname debian2
```

And, I edited the file `/etc/hosts` and substituted the string `debian1` with `debian2` in the second line

```
file /etc/hosts
127.0.0.1 localhost
127.0.1.1 debian2      <-----
...
```

To verify that the hostname was successfully changed, once again use the `hostname` or `hostnamectl` command:

```
hostnamectl
```

Now we have two debian systems on the `10.133.7.0/24` network.

1. The application

The server application.

The server application is a single threaded application that can receive multiple TCP connections simultaneously, it read from the client several bytes and replies with these bytes adding a simple string prefix.

The server application is implemented in the `tcpreplay.c` file.

The server application requires three arguments: the local port in which it waits for new connections and a string that is the name of the local network interface card as seen by the container. The third argument is the name of a environment variable. The second and third arguments are used to create the prefix.

```
./ tcpreplay.exe 61000 eth0 MYVAR
```

La variabile MYVAR, il cui nome viene passato come terzo argomento, viene cercata nell'ambiente di esecuzione dell'eseguibile ed il suo valore viene restituito a ciascun client mediante la connessione TCP che il client ha aperto con il server tcpreplay.

Notare che il valore della variabile viene istanziata nel docker-compose.yml file mettendo all'interno alcuni simboli che rappresentano il nome del container (task) in cui il server tcpreplay viene eseguito, il nome del servizio a cui appartiene quel task e, infine, il nome del nodo su cui quel task opera. **Nel momento in cui il task (container) viene creato, questi simboli vengono sostituiti con il loro contenuto e quindi la variabile MYVAR, dentro al container, come valore avrà una stringa che contiene il nome del container, del servizio e del nodo su cui il container esegue. Questa stringa viene mandata al client che quindi capisce da quale container e da quale nodo del cluster è stata gestita la sua richiesta.**

The client application.

It is a simple application that creates multiple threads, each thread establishes a TCP connection with the server and, for two times, sends a few bytes to that server and wait to receive the response, writing the responses to the standard output.

The client application is implemented in the clitcp.c file.

The client application requires three arguments: The remote ip address of the server or the server name, the local port in which the remote server is waiting for connections and, finally, the number of thread to be created (in other words, the number of simultaneous connection to be established with the server).

```
./clitcp.exe 127.0.0.1 61000 1000
```

2. DOCKER SWARM.

2.1. Docker swarm Manager initialization.

One argument only, the IP address on which the master will be reached. This assigns the local node the roles of both manager and worker.

```
docker swarm init --advertise-addr 10.133.7.101 --listen-addr 10.133.7.101
```

Swarm initialized: current node (bvktxzjwr73lsq2e0pdb5g68z) is now a manager.

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-4oj8bfuo7wk1jh0kgymq33xosmj5rv94zn3r89nfhaurdr7290-73ilqpdlphdxopip9ibc3dpt 10.133.7.101:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Note that the init command shows how you can add a **worker node** to the swarm. As you can see:

To add a worker to this swarm, run the following command:

```
docker swarm join --token \
SWMTKN-1-4oj8bfuo7wk1jh0kgymq33xosmj5rv94zn3r89nfhaudr7290-73ilqpldlphdxopip9ibc3dpt \
10.133.7.101:2377
```

where 10.133.7.101:2377 are the address and the port on which waits the swarm manager node, and where the

Note that the init command shows how you can add another manager node to the swarm. As you can see:

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

2.2. Creating a Docker registry on the swarm manager node.

I deployed a private Docker registry on the manager node so that other nodes can pull images. A container image called "registry:2" provided by dockerhub can be used to setup the service. The number 2 identifies the latest version. I assigned the name myregistry to that service.

NB: se l'host ha un nome diverso da manager allora mettere quel nome.

NB: va bene anche se specifico il ruolo invece dell'hostname, anzi forse è addirittura meglio: node.role==manager

```
docker service create --detach=true --constraint
node.role==manager --name myregistry --publish
published=5000,target=5000 registry:2
```

```
docker service create --detach=true --constraint
node.hostname==nerone --name myregistry --publish
published=5000,target=5000 registry:2
bbm62n0vny6x6ga5gxx2rx36v
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

Note that the rule -publish allows the service can be reached from outside the manager machine, overcoming the iptables firewall (run "sudo iptables -list" on manager node and find the DOCKER-INGRESS chain to see the ACCEPT tcp 5000 rule).

Very important: on each node (manager and worker), docker daemon uses HTTPS protocol to access the registry and to download the container image. To allow this protocol can work with the registry, we should create a self-signed certificate on the manager node for the registry. This is not simple. To overcome this problem, **we allow the docker daemon, on each node, to access the external registry using HTTP protocol instead of HTTPS, when accessing the IP address of the**

manager. It is a potentially dangerous configuration of the docker daemon. To modify the docker daemon configuration:

On the manager node:

```
sudo tee /etc/docker/daemon.json <<<'{ "insecure-registries":  
["127.0.0.1:5000", "10.133.7.101:5000"] }'
```

On the worker nodes:

```
sudo tee /etc/docker/daemon.json <<<'{ "insecure-registries":  
["10.133.7.101:5000"] }'
```

Then, on all nodes:

```
sudo chmod 644 /etc/docker/daemon.json  
# reload configuration of docker daemon  
sudo systemctl reload docker
```

To check if the registry works, run the following command:

```
docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
bbm62n0vny6x	registry	replicated	1/1	registry:latest	*:5000->5000/tcp

If you needs to list all the images stored in that registry, that work at the address 127.0.0.1:5000, run:

```
curl http://127.0.0.1:5000/v2/_catalog
```

Response will be in the following format:

```
{  
  "repositories": [  
    <name>,  
    ...  
  ]  
}
```

The command show the following output, because there is only one image.

```
{"repositories":["debian.iptables.ifconfig"]}
```

If you need to remove the service you can use:

```
docker service rm myregistry
```

2.3. Dockerize the server application.

In the manager node, I create a container with the server application. I create a Debian-based container with libc6 library and the gcc development environment.

I compiled the application and then remove the gcc environment, leaving the libc6 library.
Then I defined the default command the container will execute and exposed the port 61000 on which the server application wait for client connections.

Moreover, I tagged the built image with the IPAddress:port pair on which the local registry works, 127.0.0.1:5000. This allow pushing the container image to the local registry.

To do so, i created the following **docker-compose.yml** file.

Note that I used a version 3 for the docker-compose file, and I used that file for both image building and stack deploying. Note that the build subsection will be ignored at deploy time.

```
version: '3'

services:
  tcpreplayervice:
    build:
      context: .
      args:
        - VALUE_OF_DEBIAN_FRONTEND=noninteractive
      dockerfile: Dockerfile
    image: 127.0.0.1:5000/tcpreplay
    deploy:
      replicas: 1
    # replicas: 2
    placement:
      constraints:
        #OK - "node.role == manager"
        #OK - "node.hostname == nerone1"
        - "node.role == worker"
    ports:
      - 61000:61000
    environment:
      MYVAR: "NODE.ID={{ .Node.ID }} __HOSTNAME={{ .Node.Hostname }} __SERVICE={{ .Service.Name }} __TASK={{ .Task.Name }}"
```

This example sets the template of the created containers based on the service's name and the ID of the node where the container is running.

Please note: To pass to each container a parameter that contains information about the task, service and node to which the task belongs, I use a method called "creating a service using templates", as described in

https://docs.docker.com/engine/reference/commandline/service_create/#create-services-using-templates. The variable is created only when each service task is created (using the docker service create command). This command provides the ability of passing parameters that contain information on task, nodes and services, using a typical syntax of the Go language. For example `-env MIAVAR="I_am_{{.Node.Hostname}}"`

In our examples, MYVAR becomes

```
"NODE.ID=81luxmi0teedfjqnpk3j7mcxlt__HOSTNAME=debian1__SERVICE=mystack_tcpreplayservice__TASK=mystack_tcpreplayservice.1.1o0f2unq31n9xk3yktmvpprmt" on Deb1 or
"NODE.ID=a6iel6h5l3fgicp3lur2rgt1p__HOSTNAME=debian2__SERVICE=mystack_tcpreplayservice__TASK=mystack_tcpreplayservice.2.poa82sa9qc7dpyuorev4v7lwi on Deb2).
```

2.3.1. Build the server application.

To create the container image, use the docker-compose.yml file, by running:

```
docker-compose build
```

2.3.2. Save the server application container in the private local registry.

To upload the container image to the private registry, run:

```
docker-compose push
```

2.4 Deploy the server application container in the single-node swarm.

To run the application in the swarm, when creating a new stack in swarm mode, docker requires the version number 3 (or above) of the compose file and does not allow the build section. Fortunately, the build section will be ignored.

I used the **docker-compose.yml** manifest file cited above.

To run the application in the manager node that, in my example, is the only worker of the swarm, you must create the stack, and compose will create the services in that stack. Create and run the application using:

```
docker stack deploy --detach=true --compose-file docker-compose.yml mystack
```

where mystack is the name I want to assign to my server in this run.

The docker stack command provides five sub-commands:

deploy	Deploy a new stack or update an existing stack
ls	List stacks
ps	List the tasks in the stack
rm	Remove one or more stacks
services	List the services in the stack

To verify that the stack has been created, run the command:

```
docker stack ls
```

NAME	SERVICES	ORCHESTRATOR
mystack	1	Swarm

To verify the services running in the stack, run the command:

docker stack services mystack

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
qh8snycqzxcd	mystack_tcprelayservice	replicated	1/1	127.0.0.1:5000/tcprelay:latest	*:61000->61000/tcp

As you can see, the output show **1/1 replicas** because the only worker node in the manager.

docker stack ps mystack

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
pdetqfjgjrj	mystack_tcprelayservice.1	127.0.0.1:5000/tcprelay:latest	debian1	Running	Running 2 hours ago		
cacjapfzadlu	_ mystack_tcprelayservice.1	127.0.0.1:5000/tcprelay:latest	debian1	Shutdown	Shutdown 2 hours ago		
6rqsequ8ggvq	_ mystack_tcprelayservice.1	127.0.0.1:5000/tcprelay:latest	debian1	Shutdown	Rejected 2 hours ago	"No such image: 127.0.0.1:5000..."	

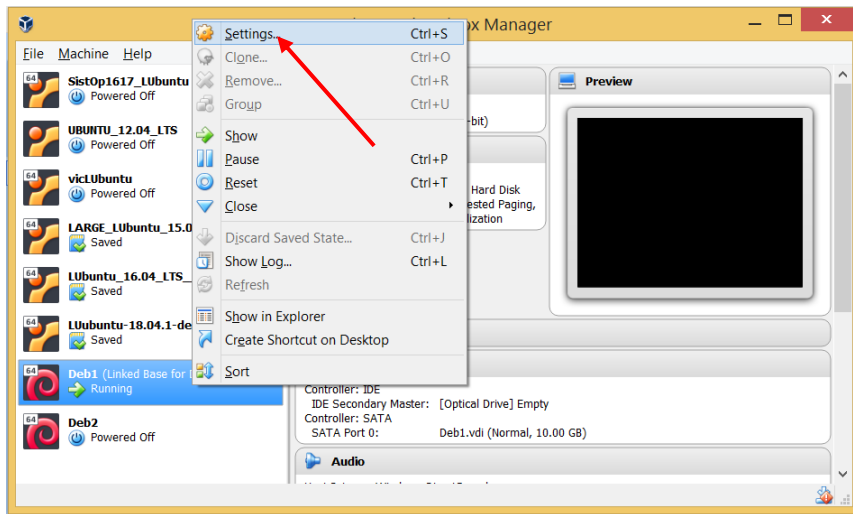
2.5. Using the service through the manager node.

2.5.1. Forwarding port to the manager virtual machine.

The deb1 machine, in which the docker manager works, can be a phisycal host with two network interface cards. The second NIC connects the Deb1 machine with the second Deb2 machine. The tcprelay service waits for connections on its 61000 tcp port. The "mystack" stack exposes that 61000 tcp port on the edge of the Deb1 manager node. Thus, the tcprelay service can be reached through the 61000 tcp port of the manager node. The docker engine on the manager node will redirect each connection to a worker node of the swarm, using the second internal NIC.

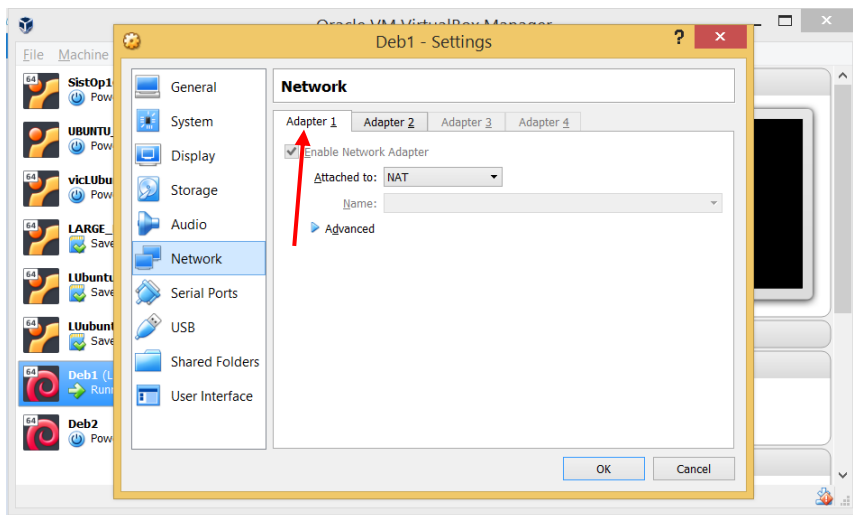
In our experimental scenario, instead, the deb1 machine is a virtual machine provided by virtualbox in a phisycal host. A NAT isolates the Deb1 machine and avoid an external client can reach the 61000 tcp port exposed by the "mystack" stack at the edge of the Deb1 manager node. In order to allow an external client can reach the 61000 tcp port of the Deb1 manager node, we must configure virtualbox virtual machine of Deb1 to forward its 61000 tcp port.

To forward ports in VirtualBox, first open the Deb1 manager virtual machine's settings window by selecting the Settings option in the menu.

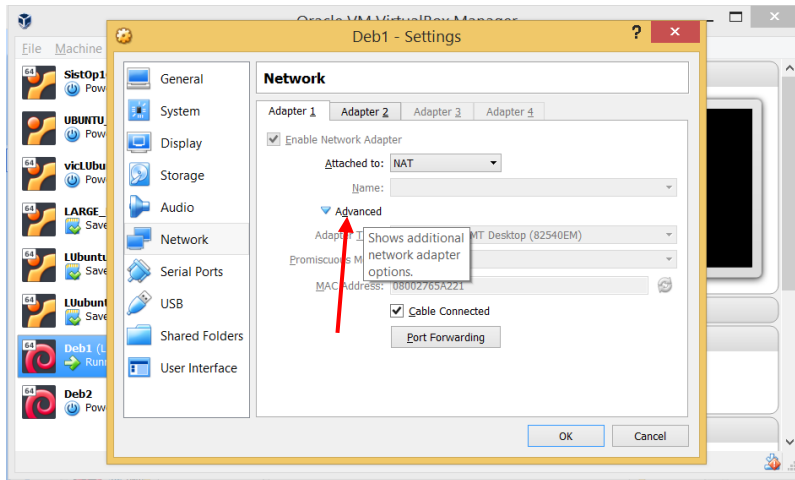


Select the Network panel in the virtual machine's configuration window.

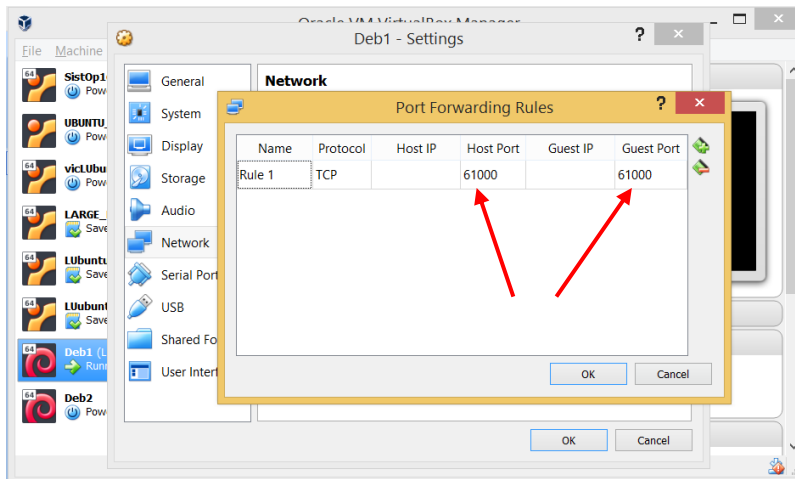
Select the network adapter 1 (the NAT type network) that connects the node to the physical host.



Expand the Advanced section and click the Port Forwarding button. Note that this button is only active if you're using a NAT network type - you only need to forward ports if you're using a NAT.



Use VirtualBox's Port Forwarding Rules window to forward ports. Select the + button to add a port forwarding rule.



Insert 61000 on both "Host Port" and "Guest Port". You don't have to specify any IP addresses - those two fields are optional. Thus, leave blank space on both "Host IP" and "Guest IP". Note: While you don't have to enter any IP details, leaving the Host IP box blank will make VirtualBox listen on 0.0.0.0—in other words, it will accept all traffic from the local network and forward it to your virtual machine. Enter 127.0.0.1 in the Host IP box and VirtualBox will only accept traffic originating on your computer—in other words, on the host operating system. Select Ok to confirm your choice.

2.5.2. Test the service through the manager node.

In our scenario, the deb1 machine, in which the docker manager works, is a virtual machine hosted in a physical host. The manager node's virtual machine forwards its 61000 tcp port that becomes reachable,

from outside the Deb1 machine, using the IP address of the physical host.

In our scenario, the clients are deployed on a third ubuntu virtual machine that operates in a separated network. This ubuntu system executes the client application and connects the manager node tcprelay service using the IP address of the physical host and the forwarded 6100 tcp port. Alternatively, the client application can connect the manager node tcprelay service using the IP address of the docker0 bridge as seen by the physical host (Ethernet adapter VirtualBox Host-Only Network: IPv4 Address: 192.168.56.1).

On the third client machine, run

```
./clitcp.exe 192.168.56.1 61000 100
```

2.6. Adding another node to the swarm.

I started the other deb2 debian virtual machine.

I added that deb2 virtual machine as node in the swarm, running:

```
docker swarm join --token SWMTKN-1-  
4oj8bfuo7wk1jh0kgymq33xosmj5rv94zn3r89nfhaudr7290-  
73ilqpldlphdxopip9ibc3dpt 10.133.7.101:2377
```

This node joined a swarm as a worker.

To show all the nodes of the swarm, on the manager node run the command:

```
docker node ls
```

But, this added worker node, actually does not provide support for the tcprelay service. In fact, in the manager node, You can check that the number of replicas is 1/1.

```
docker stack services mystack
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
qh8snycqzxcd	mystack_tcprelayservice	replicated	1/1	127.0.0.1:5000/tcprelay:latest	*:61000->61000/tcp

2.7. Run the application on both the worker nodes.

To run my application on both the nodes, I ran Docker Swarm's **scale** command on the manager node:

```
docker service scale mystack_tcprelayservice=2
```

```
mystack_tcprelayservice scaled to 2  
overall progress: 2 out of 2 tasks  
1/2: running  
2/2: running  
verify: Service converged
```

This command defines the number of containers that provides the service. If more worker nodes are available, the containers are deployed on the different workers.

And now, in the manager node, You can check that the number of replicas is 2/2.

docker stack services mystack

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
qh8snycqzxcd	mystack_tcpreplayservice	replicated	2/2	127.0.0.1:5000/tcpreplay:latest	*:61000->61000/tcp

And, On the new worker node, the new container showed up:

docker container ls

CONTAINER ID	IMAGE	COMMAND	CREATED
aa070bf8e347	127.0.0.1:5000/tcpreplay:latest	"/bin/sh -c './tcpre..."	4 minutes ago
minutes	61000/tcp	mystack_tcpreplayservice.2.mnife7wsqrqj4qtas6tx9vao	Up 4

More simply, to show where the tasks (containers) are located, on the manager node run the command:

docker stack ps mystack

ID	NAME	IMAGE	NODE	DESIRED_STATE	CURRENT STATE
ERROR	PORTS				
cxxxuedmb4u	mystack_tcpreplayservice.1	127.0.0.1:5000/tcpreplay:latest	manager	Running	Running 14 minutes ago
m5m81noyszah	mystack_tcpreplayservice.2	127.0.0.1:5000/tcpreplay:latest	worker01	Running	Running 9 minutes ago

2.8. Testing load balancing among worker nodes.

In the manager node, run the 100-thread client:
started the other deb2 debian virtual machine.

`./clitcp.exe 192.168.56.1 61000 100`

And, on each worker node, shows the running container
(mystack_tcpreplayservice.4.fabgsz8cavj466ahhw54271lll etc etc etc):
`docker ps -a`

2.9. Detach a worker node from the swarm.

On the worker node, run the following command to leave the swarm:

docker swarm leave

On the manager node, run the following command to leave the swarm in spite of the following warning :

Error response from daemon: You are attempting to leave the swarm on a node that is participating as a manager. Removing the last manager erases all current state of the swarm. Use `--force` to ignore this message.

docker swarm leave --force

In this way, the node returns to the single-node mode (no swarm mode).

2.10. Force the manager not to work as a worker

If you want, you can force the manager node to work as manager only.

```
docker node update --availability drain node-1
```

2.11. Bring the stack down with docker stack rm:

On the manager node, run the command:

```
docker stack rm mystack
```

where `mystack_tcpreplayservice` is the name of the service to be removed. This command stops the service and removes the running container in all the worker nodes, and removes the stack.

2.12. Stop a service on all worker nodes

If you need, you can stop a single service only.

On the manager node, run the command:

```
docker service rm mystack_tcpreplayservice
```

where `mystack_tcpreplayservice` is the name of the service to be removed. This command stops the service and removes the running container in all the worker nodes, and removes the stack.

And Finally:

2.13. Bring the registry down with docker service rm:

```
docker service rm myregistry
```

And verify there are no more services

```
docker service ls
```

2.14. Switch the manager node to normal mode (no swarm mode)

```
docker swarm leave --force
```

In this way, the node returns to the single-node mode.

For more information, see:

<https://docs.docker.com/engine/swarm/>